Dynamic Programming

1 Introduction

Dynamic Programming (DP) is an algorithmic technique used to solve problems by breaking them into smaller overlapping subproblems and storing their solutions to avoid repeated computation.

2 Characteristics of Dynamic Programming

2.1 Optimal Substructure

A problem has optimal substructure if its optimal solution can be constructed from optimal solutions of its subproblems.

2.2 Overlapping Subproblems

The same subproblems are solved multiple times in a recursive solution.

3 Dynamic Programming vs Divide and Conquer

Divide and Conquer	Dynamic Programming
Independent subproblems	Overlapping subproblems
No storage of results	Stores subproblem results
Example: Merge Sort	Example: Fibonacci

4 Approaches to Dynamic Programming

4.1 Top-Down Approach (Memoization)

Uses recursion with a memory table to store previously computed results.

4.2 Bottom-Up Approach (Tabulation)

Builds the solution iteratively starting from base cases without recursion.

5 Steps to Solve a DP Problem

1. Define the DP state
2. Derive the recurrence relation
3. Identify base cases
4. Choose computation order
5. Optimize space if possible

6 Classic DP Problems

6.1 Fibonacci Series

State: $dp[n]$ = nth Fibonacci number

Recurrence:

$$dp[n] = dp[n-1] + dp[n-2]$$

Time Complexity: $O(n)$

6.2 0/1 Knapsack Problem

State: $dp[i][w]$ = maximum value using first i items with capacity w

Recurrence:

$$dp[i][w] = \max(dp[i-1][w],\ value_i + dp[i-1][w-w_i])$$

Time Complexity: $O(nW)$

6.3 Longest Common Subsequence (LCS)

State: $dp[i][j]$ = length of LCS of first i characters of string A and first j of string B

7 Applications

- Compiler design
- Network routing
- Control systems
- Bioinformatics
- Machine learning

8 Advantages and Limitations

8.1 Advantages

- Reduces time complexity
- Guarantees optimal solution

8.2 Limitations

- High memory usage
- Complex state design

9 Summary

Dynamic Programming is a powerful technique for solving optimization problems by storing solutions to overlapping subproblems and building the final solution efficiently.